

CO2-AT2- Scenario Based Questions

Name: Shaik Azeem

Reg.no: 192472307

Course code: CSA0712

Course name: Computer Network

Scenario 1: Noisy Channel and Frame Loss

Q1. How does a Stop-and-Wait ARQ protocol react when Frame 4 goes missing?

- The sender transmits Frame 4 and waits for an ACK.
- Since Frame 4 is lost, the receiver does not receive it and cannot send an ACK.
- The sender waits until the timeout expires.
- After the timeout, the sender retransmits Frame 4.

Q2. What happens to Frames 5 and 6 if Stop-and-Wait is used?

- Frames 5 and 6 are not transmitted immediately.
- Stop-and-Wait sends only one frame at a time.
- The sender must successfully receive the ACK for Frame 4 before sending Frame 5.
- Therefore, Frames 5 and 6 remain waiting.

Q3. How does Go-Back-N ARQ handle the lost Frame 4 compared to Selective Repeat ARQ?

GO-BACK-N ARQ	Selective Repeat ARQ
Receiver discards Frames 5, 6, 7... because Frame 4 is missing.	Receiver accepts and buffers Frames 5, 6, 7...
Sender retransmits Frame 4 and all following frames.	Sender retransmits only Frame 4.
More bandwidth is wasted.	Bandwidth is used efficiently.

Scenario 2: MAC Sublayer and Collisions

Q1. What network event happens when both signals meet on the wire?

- A collision occurs.
- Both signals interfere with each other, making the transmitted data unreadable.

Q2. How does CSMA/CD detect this event?

- Each computer listens to the network while transmitting.
- If the transmitted signal differs from the received signal, it detects a collision.
- The stations stop transmitting immediately and send a jam signal so all devices know a collision occurred.

Q3. Why do A and B wait for a random time before retransmitting?

- If both retransmit immediately, another collision may occur.
- Therefore, each waits for a random backoff time using the Binary Exponential Backoff Algorithm.
- This reduces the chance of repeated collisions.

Scenario 3: Flow Control and Buffer Overflow

Q1. What problem happens if the server sends frames faster than the printer can clear its buffer?

- The printer's buffer becomes full.
- New incoming frames are dropped.
- This causes buffer overflow, resulting in data loss.

Q2. How does a sliding window flow control protocol prevent overflow?

- The receiver advertises how many frames it can accept.
- The sender transmits only within the allowed window size.
- As the printer processes data and sends ACKs, the window slides forward.
- This prevents sending more data than the printer can store.

Q3. What role does the window size field in the acknowledgement frame play?

- It tells the sender the available buffer space at the receiver.
- The sender adjusts its transmission rate according to this value.
- Smaller window → slower transmission.
- Larger window → faster transmission.

Scenario 4: Error Detection with CRC

Q1. How many CRC bits are appended?

Generator Polynomial:

$$G(x) = x^3 + x + 1 \rightarrow 1011$$

Data: 110101

Generator length = 4 bits

$$\text{CRC bits} = 4 - 1 = 3 \text{ bits}$$

Q2. Modulo-2 Division

Q4:

Generator Polynomial

$$G(x) = x^3 + x + 1$$

$$x^3 + x^2 + x + 1$$

$$1 \ 0 \ 1 \ 1$$

Binary: 1011

Data: 110101

1. Generator Polynomial = 1011

length = 4 bits

No. of CRC bits = $4 - 1 = 3$ bits

2. Perform modulo-2 division:

$$\begin{array}{r} 1011 \overline{) 110101000} \\ \underline{1011} \\ 011001000 \\ \underline{1011} \\ 01111000 \\ \underline{1011} \\ 0100000 \\ \underline{1011} \\ 001100 \\ \underline{1011} \\ 0111 \end{array}$$

$$\begin{array}{r} 110101000 \\ \underline{111} \\ \hline 11010111 \end{array}$$

Data: 110101000

CRC bit = 111

Transmitted sequence
= 11010111

Q3. If the first two bits are flipped during transmission, how does the receiver detect it?

The Original Data:

110101111

The Transmitted Data:

000101111

The receiver performs the same CRC division using 1011.

- If the remainder is 000, the frame is correct.
- Since two bits changed, the remainder is non-zero.
- Therefore, the receiver concludes that the frame is corrupted and discards it (or requests retransmission if ARQ is used).

Scenario 5: Framing and Byte Stuffing

Q1. Why does the receiver think the frame ended early?

- The flag byte 0x7E marks the start and end of a frame.
- If 0x7E appears inside the payload, the receiver mistakes it for the end-of-frame delimiter.
- This causes the frame to end prematurely.

Q2. How does byte stuffing solve this issue?

- The sender inserts an escape byte (0x7D) before every occurrence of:
 - 0x7E (flag)
 - 0x7D (escape)
- The receiver removes the escape byte during decoding and reconstructs the original data.
- The sender scans the payload before transmission. Whenever it finds a byte that matches the flag byte (0x7E) or the escape byte itself it inserts a special escape byte (commonly 0x7D) immediately before it.

- The receiver, upon seeing an escape byte, knows the *next* byte is actual data (not a flag) and removes the escape byte during destuffing, restoring the original payload.

Q3. Show the transformation

Assume:

- Flag = **0x7E**
- Escape = **0x7D**
- Data byte **0x7E** (inside payload) → sender transmits **0x7D 0x7E** (escape byte + original byte, sometimes with the byte XORed, e.g., **0x7D 0x5E**, depending on the exact scheme but conceptually: escape + the byte itself)
- Escape byte **0x7D** (if it appears in payload) → sender transmits **0x7D 0x7D** (it escapes itself too)

Original Byte	Sent Byte(s)
0x7E	0x7D 0x5E
0x7D	0x7D 0x5D